

Material de Patrones de Diseño GoF para la Técnica de Aronson

Prof. Daniel San Martín

9 de abril de 2025

1. Patrones Estructurales

El objetivo de los patrones de estructuración es facilitar la independencia de la interfaz de un objeto o de un conjunto de objetos respecto de su implementación. En el caso de un conjunto de objetos, se trata también de hacer que esta interfaz sea independiente de la jerarquía de clases y de la composición de los objetos.

Proporcionando interfaces, los patrones de estructuración encapsulan la composición de objetos, aumentan el nivel de abstracción del sistema de forma similar a como los patrones de creación encapsulan la creación de objetos. Los patrones de estructuración ponen de relieve las interfaces.

La encapsulación de la composición no se realiza estructurando el objeto en sí mismo sino transfiriendo esta estructuración a un segundo objeto. Éste queda íntimamente ligado al primero. Esta transferencia de estructuración significa que el primer objeto posee la interfaz de cara a los clientes y administra la relación con el segundo objeto que gestiona la composición y no tiene ninguna interfaz con los clientes externos.

Esta realización ofrece otra mejora que es la flexibilidad en la composición, la cual puede modificarse de manera dinámica. En efecto, es sencillo sustituir un objeto por otro siempre que sea de la misma clase o que respete la misma interfaz. Los patrones **Composite**, **Decorator** y **Bridge** son un buen ejemplo de este mecanismo.

1.1. Proxy

El patrón Proxy tiene como objetivo el diseño de un objeto que sustituye a otro objeto (el sujeto) y que controla el acceso. El objeto que realiza la sustitución posee la misma interfaz que el sujeto, volviendo la sustitución transparente de cara a los clientes.

1.1.1. Ejemplo

Queremos ofrecer para cada vehículo del catálogo la posibilidad de visualizar un pequeño vídeo de presentación del vehículo. Un clic sobre la fotografía de la presentación del vehículo permitirá reproducir este vídeo.

Una página del catálogo contiene numerosos vehículos y es muy pesado guardar en memoria todos los objetos de animación, pues los vídeos necesitan gran cantidad de memoria, y su transferencia a través de la red toma bastante tiempo.

El patrón Proxy ofrece una solución a este problema difiriendo la creación de los sujetos hasta el momento en que el sistema tiene necesidad de ellos, en este caso tras un clic en la fotografía del vehículo.

Esta solución aporta dos ventajas:

- La página del catálogo se carga mucho más rápidamente, sobre todo si tiene que cargarse Sólo aquellos a través vídeos de una que red van como a visualizarse Internet.
- Sólo aquellos vídeos que van a visualizarse se crean, cargan y reproducen.

El objeto **fotografía** se llama el proxy del **vídeo**. Procede a la creación del sujeto únicamente tras haber hecho clic en ella. Posee la misma interfaz que el objeto **vídeo**. La Figura 16.1 muestra el diagrama de clases correspondiente. La clase del **proxy**, **AnimaciónProxy**, y la clase del vídeo, **Vídeo**, implementan ambas la misma interfaz, a saber **Animación**.

Cuando el proxy recibe el mensaje **dibuja**, muestra el vídeo si ha sido creado y cargado. Cuando el proxy recibe el mensaje **clic**, reproduce el vídeo después de haberlo creado y cargado previamente.

1.1.2. Estructura

La Figura 2 ilustra la estructura genérica del patrón. Conviene observar que los métodos del proxy tienen dos comportamientos posibles cuando el sujeto real no ha sido creado: o bien crean el sujeto real y a continuación le delegan el mensaje (es el caso del método **clic** del ejemplo), o bien ejecutan un código de sustitución (es el caso del método **dibuja** del ejemplo).

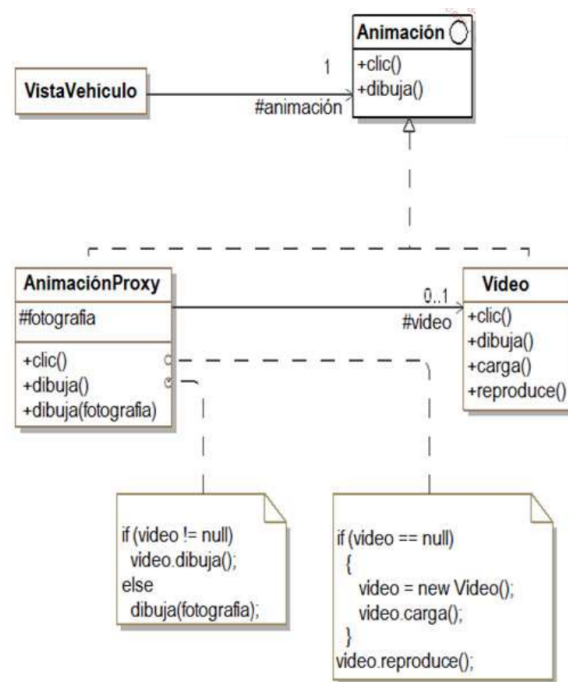


Figura 1: El patrón Proxy aplicado a la visualización de animaciones

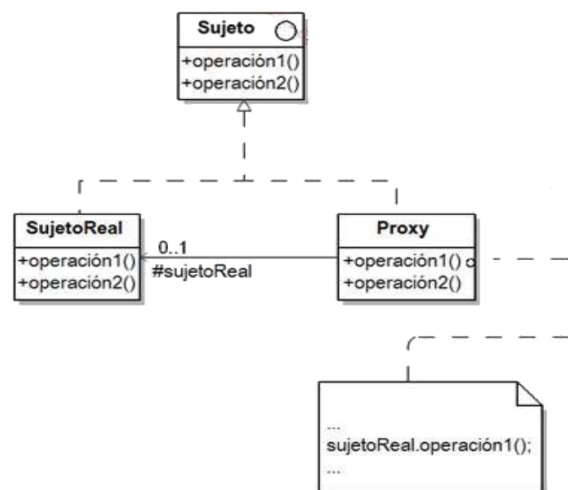


Figura 2: Estructura del patrón Proxy

1.1.3. Participantes

Los participantes del patrón son los siguientes:

- **Sujeto** (Animación) es la interfaz común al proxy y al sujeto real.
- **SujetoReal** (Video) es el objeto que el proxy controla y representa.
- **Proxy** (AnimaciónProxy) es el objeto que se sustituye por el sujeto real. Posee una interfaz idéntica a este último (interfaz Sujeto). Se encarga de crear y de destruir al sujeto real y de delegarle los mensajes.

1.1.4. Colaboraciones

El proxy recibe las llamadas del cliente en lugar del sujeto real. Cuando lo juzga apropiado, delega estos mensajes en el sujeto real. Debe, en este caso, crear previamente el sujeto real si no está creado ya.

1.1.5. Dominios de aplicación

Los proxys son muy útiles en programación orientada a objetos. Existen distintos tipos de proxy. Vamos a ilustrar tres:

- Proxy virtual: permite crear un objeto de tamaño importante en el momento adecuado.
- Proxy remoto: permite acceder a un objeto ejecutándose en otro entorno. Este tipo de proxy se implementa en sistemas de objetos remotos (CORBA, Java RMI).
- Proxy de protección: permite securizar el acceso a un objeto, por ejemplo mediante técnicas de autenticación.

1.1.6. Ejemplo de Java

Retomamos nuestro ejemplo en Java. El código fuente de la interfaz `Animacion` aparece a continuación.

```
1 public interface Animacion {
2     void dibuja();
3     void clic();
4 }
```

El código fuente Java de la clase `Video` que implementa esta interfaz aparece a continuación. En el marco de la simulación cada método muestra simplemente un mensaje, excepto el método `clic` que no realiza ninguna acción.

```
1 public class Video implements Animacion {
2
3     public void clic() { }
4
5     public void dibuja() {
6         System.out.println("Mostrar el video");
7     }
8
9     public void carga() {
10        System.out.println("Cargar el video");
11    }
12
13    public void reproduce() {
14        System.out.println("Reproducir el video");
15    }
16 }
```

El código fuente del proxy, y por tanto de la clase `AnimacionProxy`, aparece a continuación. El código de los métodos corresponde al especificado en el diagrama de clases de la Figura 2.

```
1 public class AnimacionProxy implements Animacion {
2     protected Video video = null;
3     protected String foto = "mostrar la foto";
4
5     public void clic() {
6         if (video == null) {
7             video = new Video();
8             video.carga();
9         }
10        video.reproduce();
11    }
12    public void dibuja() {
13        if (video != null)
14            video.dibuja();
15        else
16            dibuja(foto);
17    }
18    public void dibuja(String foto) {
19        System.out.println(foto);
20    }
21 }
```

Por último, la clase `VistaVehiculo` que representa al programa principal se escribe de la siguiente manera.

```
1 public class VistaVehiculo {
2     public static void main(String[] args) {
3         Animacion animacion = new AnimacionProxy();
4         animacion.dibuja();
5         animacion.clic();
6         animacion.dibuja();
7     }
8 }
```

La ejecución de este programa muestra la diferencia de comportamiento del método `dibuja` del proxy según el método `clic` haya sido invocado previamente o no.

mostrar la foto
Cargar el vídeo
Reproducir el vídeo
Mostrar el vídeo

1.2. Adapter

1.2.1. Descripción

El objetivo del patrón Adapter es convertir la interfaz de una clase existente en la interfaz esperada por los clientes también existentes de modo que puedan trabajar de manera conjunta. Se trata de conferir a una clase existente una nueva interfaz para responder a las necesidades de los clientes.

1.2.2. Ejemplo

El servidor web del sistema de venta de vehículos crea y administra los documentos destinados a los clientes. La interfaz Documento se ha definido para realizar esta gestión. La Figura 3 muestra su representación UML así como los tres métodos `setContenido`, `dibuja` e `imprime`. Se ha realizado una primera clase de implementación de esta interfaz: la clase `DocumentoHtml` que implementa estos tres métodos. Los objetos clientes de esta interfaz y esta clase cliente ya se han diseñado.

Por otro lado, la agregación de documentos PDF supone un problema, pues se trata de documentos más complejos de construir y de administrar que los documentos HTML. Para ello se ha escogido un producto del mercado, aunque su interfaz no se corresponde con la interfaz Documento. La Figura 3 muestra el componente `ComponentePdf` cuya interfaz incluye más métodos y la nomenclatura es bien diferente (con el prefijo pdf).

El patrón Adapter proporciona una solución que consiste en crear la clase `DocumentoPdf` que implemente la interfaz Documento y posea una asociación con `ComponentePdf`. La implementación de los tres métodos de la interfaz Documento consiste en delegar correctamente las llamadas al componente PDF. Esta solución se muestra en la Figura 3, el código de los métodos se detalla con ayuda de notas.

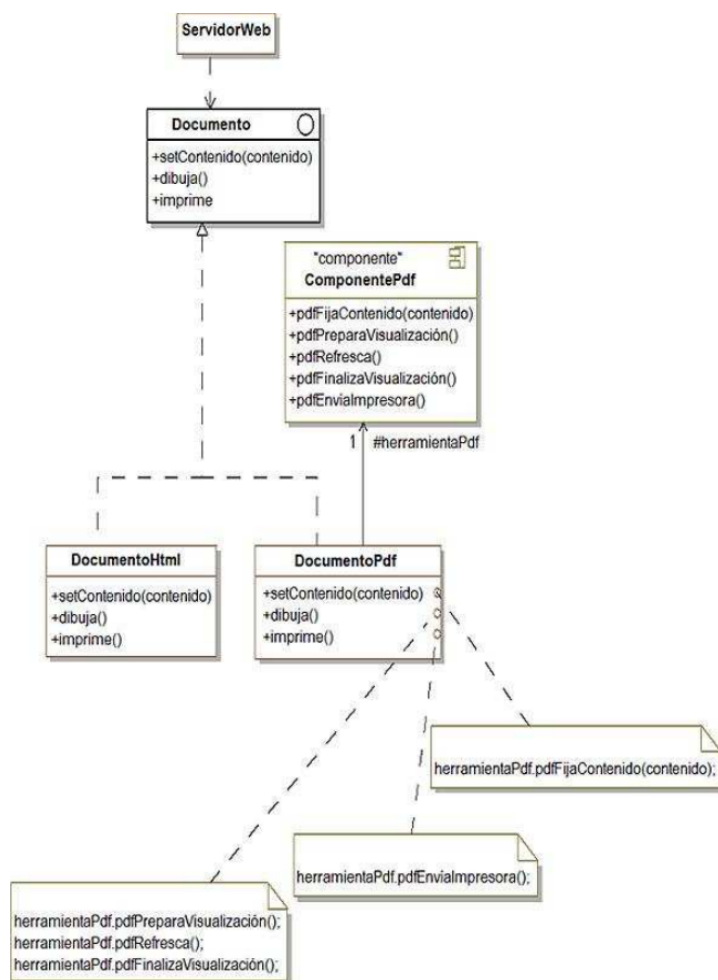


Figura 3: El patrón Adapter aplicado a un componente de documentos PDF

1.2.3. Estructura

La Figure 4 detalla la estructura genérica del patrón.

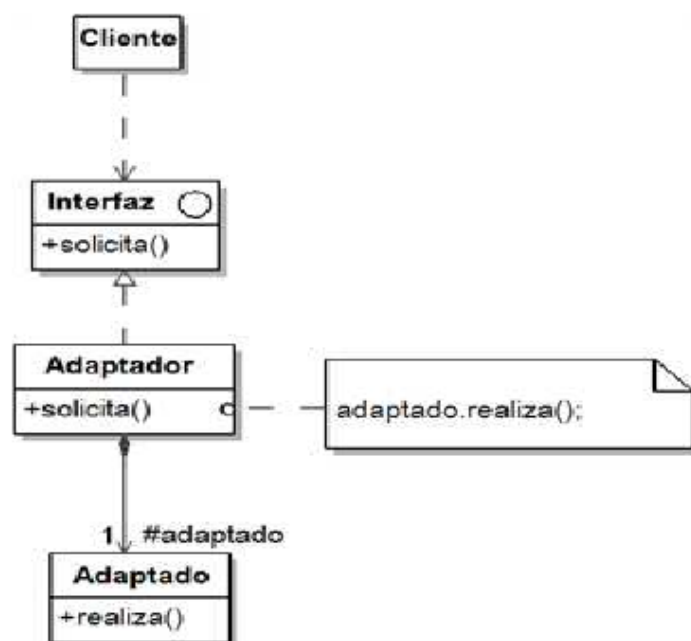


Figura 4: Estructura del patrón Adapter

1.2.4. Participantes

Los participantes del patrón son los siguientes:

- **Interfaz** (Documento) incluye la firma de los métodos del objeto.
- **Cliente** (ServidorWeb) interactúa con los objetos respondiendo a la interfaz **Interfaz**.
- **Adaptador** (DocumentoPdf) implementa los métodos de la interfaz **Interfaz** invocando a los métodos del objeto adaptado.
- **Adaptado** (ComponentePdf) incluye el objeto cuya interfaz ha sido adaptada para corresponder a la interfaz **Interfaz**.

1.2.5. Colaboraciones

El cliente invoca el método solicitud del adaptador que, en consecuencia, interactúa con el objeto adaptado invocando el método `realiza()`.

1.2.6. Dominios de aplicación

El patrón se utiliza en los siguientes casos:

- Para integrar en el sistema un objeto cuya interfaz no se corresponde con la interfaz requerida en el interior de este sistema.
- Para proveer interfaces múltiples a un objeto en su etapa de diseño.

1.2.7. Ejemplo Java

A continuación se presenta el código del ejemplo escrito en Java.

```
1 public interface Documento {
2     void setContenido(String contenido);
3     void dibuja();
4     void imprime();
5 }
6
```

```

7 // La clase DocumentoHtml es el ejemplo de clase que implementa
8 // la interfaz Documento.
9
10 public class DocumentoHtml implements Documento {
11     protected String contenido;
12     public void setContenido(String contenido) {
13         this.contenido = contenido;
14     }
15     public void dibuja() {
16         System.out.println("Dibuja el documento HTML: " + contenido);
17     }
18     public void imprime() {
19         System.out.println("Imprime el documento HTML: " + contenido);
20     }
21 }

```

La clase `ComponentePdf` representa el componente existente que se quiere integrar en la aplicación. Su diseño es independiente de la aplicación y, en particular, de la interfaz `Documento`. Esta clase tendrá que adaptarse a continuación.

```

1 public class ComponentePdf {
2     protected String contenido;
3     public void pdfFijaContenido(String contenido) {
4         this.contenido = contenido;
5     }
6     public void pdfPreparaVisualizacion() {
7         System.out.println("Visualiza PDF: Comienzo");
8     }
9     public void pdfRefresca() {
10        System.out.println("Visualiza contenido PDF: " + contenido);
11    }
12    public void pdfFinalizaVisualizacion() {
13        System.out.println("Visualiza PDF: Fin");
14    }
15    public void pdfEnviaImpresora() {
16        System.out.println("Impresion PDF: " + contenido);
17    }
18 }

```

La clase `DocumentoPdf` representa el adaptador. Está asociada a la clase `ComponentePdf` mediante el atributo `herramientaPdf` que se asocia con el objeto adaptado.

Implementa la interfaz `Documento` y cada uno de sus métodos invoca a los métodos necesarios del objeto adaptado para realizar la adaptación entre ambas interfaces.

```

1 public class DocumentoPdf implements Documento {
2
3     protected ComponentePdf herramientaPdf = new ComponentePdf();
4
5     public void setContenido(String contenido) {
6         herramientaPdf.pdfFijaContenido(contenido);
7     }
8
9     public void dibuja() {
10        herramientaPdf.pdfPreparaVisualizacion();
11        herramientaPdf.pdfRefresca();
12        herramientaPdf.pdfFinalizaVisualizacion();
13    }
14
15    public void imprime() {
16        herramientaPdf.pdfEnviaImpresora();
17    }
18 }

```

El programa principal se corresponde con la clase `ServidorWeb` que crea un documento HTML, fija el contenido y a continuación lo dibuja. A continuación, el programa realiza las mismas acciones con un documento PDF.

```

1 public class ServidorWeb {
2     public static void main(String[] args) {
3         Documento documento1, documento2;
4         documento1 = new DocumentoHtml();
5         documento1.setContenido("Hello");
6         documento1.dibuja();
7         System.out.println();
8         documento2 = new DocumentoPdf();
9         documento2.setContenido("Hola");
10        documento2.dibuja();
11    }

```

La ejecución de este programa principal da el resultado siguiente.

```
Dibuja documento HTML: Hello
Visualiza PDF: Comienzo
Visualiza contenido PDF: Hola
Visualiza PDF: Fin
```


1.3. Facade

1.3.1. Descripción

El objetivo del patrón Facade es agrupar las interfaces de un conjunto de objetos en una interfaz unificada volviendo a este conjunto más fácil de usar por parte de un cliente.

El patrón Facade encapsula la interfaz de cada objeto considerada como interfaz de bajo nivel en una interfaz única de nivel más elevado. La construcción de la interfaz unificada puede necesitar implementar métodos destinados a componer las interfaces de bajo nivel.

1.3.2. Ejemplo

Queremos ofrecer la posibilidad de acceder al sistema de venta de vehículos como servicio web. El sistema está arquitecturizado bajo la forma de un conjunto de componentes que poseen su propia interfaz como:

- El componente Catálogo.
- El componente GestiónDocumento.
- El componente RecogidaVehículo.

Es posible dar acceso al conjunto de la interfaz de estos componentes a los clientes del servicio web, aunque esta posibilidad presenta dos inconvenientes principales:

- Algunas funcionalidades no las utilizan los clientes del servicio web, como por ejemplo las funcionalidades de visualización del catálogo.
- La arquitectura interna del sistema responde a las exigencia de modularidad y evolución que no forman parte de las necesidades de los clientes del servicio web, para los que estas exigencias suponen una complejidad inútil.

El patrón Facade resuelve este problema proporcionando una interfaz unificada más sencilla y con un nivel de abstracción más elevado. Una clase se encarga de implementar esta interfaz unificada utilizando los componentes del sistema.

Esta solución se ilustra en la Figura 5. La clase `WebServiceAuto` ofrece una interfaz a los clientes del servicio web. Esta clase y su interfaz constituyen una fachada de cara a los clientes.

La interfaz de la clase `WebServiceAuto` está constituida por el método `buscaVehiculos(precioMedio, desviaciónMax)` cuyo código consiste en invocar al método `buscaVehiculos(precioMin, precioMax)` del catálogo adaptando el valor de los argumentos de este método en función del precio medio y de la desviación máxima.

Conviene observar que si bien la idea del patrón es construir una interfaz de más alto nivel de abstracción, nada nos impide proporcionar en la fachada accesos directos a ciertos métodos de los componentes del sistema.

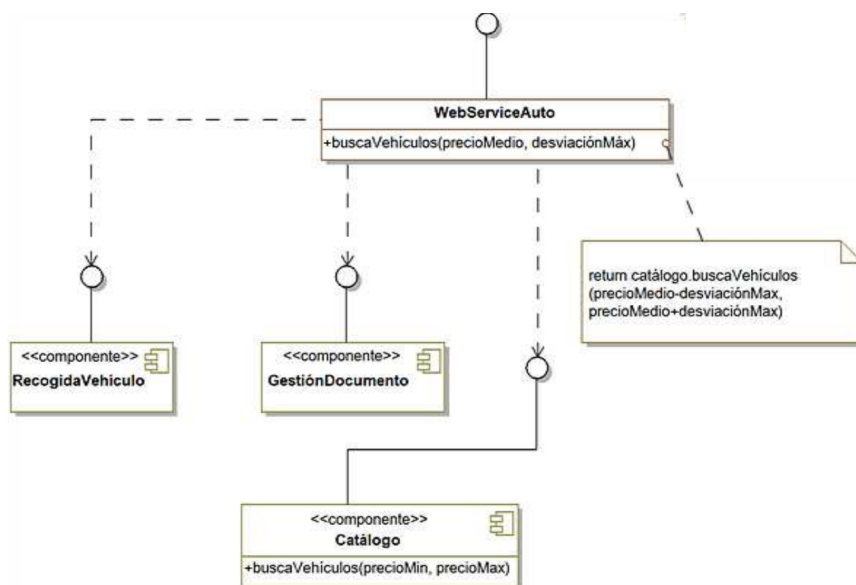


Figura 5: Aplicación del patrón Facade a la implementación del servicio Web del sistema de venta de vehículos

1.3.3. Estructura

La Figura 6 detalla la estructura genérica del patrón.

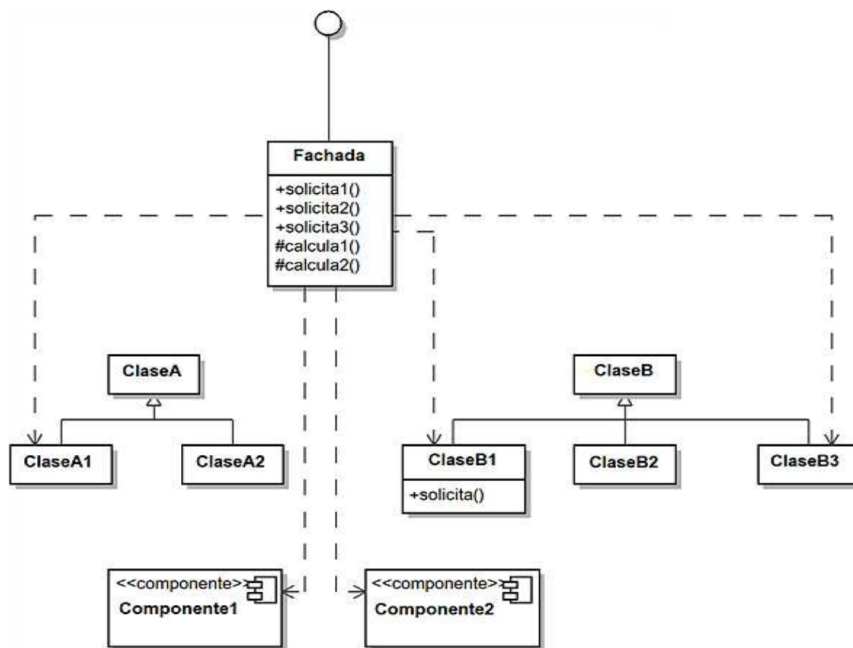


Figura 6: Estructura del patrón Facade

1.3.4. Participantes

Los participantes del patrón son los siguientes:

- Fachada (`WebServiceAuto`) y su interfaz constituyen la parte abstracta expuesta a los clientes del sistema. Esta clase posee referencias hacia las clases y componentes que forman el sistema y cuyos métodos se utilizan en la fachada para implementar la interfaz unificada.
- Las clases y componentes del sistema (`RecogidaVehículo`, `GestiónDocumento` y `Catálogo`) implementan las funcionalidades del sistema y responden a las consultas de la fachada. No necesitan a la fachada para trabajar.

1.3.5. Colaboraciones

Los clientes se comunican con el sistema a través de la fachada que se encarga, de forma interna, de invocar a las clases y los componentes del sistema. La fachada no puede limitarse a transmitir las invocaciones. También debe realizar la adaptación entre su interfaz y la interfaz de los objetos del sistema mediante código específico. Los clientes que utilizan la fachada no deben acceder directamente a los objetos del sistema.

1.3.6. Dominios de aplicación

El patrón se utiliza en los siguientes casos:

- Para proveer una interfaz simple de un sistema complejo. La arquitectura de un sistema puede estar basada en numerosas clases pequeñas, que ofrecen una buena modularidad y capacidad de evolución. No obstante estas propiedades tan estupendas no interesan en absoluto a los clientes, que sólo necesitan un acceso simple que responda a sus exigencias.
- Para dividir un sistema en subsistemas, la comunicación entre subsistemas se define de forma abstracta a su implementación gracias a las fachadas.
- Para sistematizar la encapsulación de la implementación de un sistema de cara al exterior.

1.3.7. Ejemplo en Java

Retomamos el ejemplo del servicio web que vamos a simular con ayuda de un pequeño programa escrito en Java. Se muestra en primer lugar el código fuente de los componentes del sistema, comenzando por la clase `ComponenteCatalogo` y su interfaz `Catalogo`.

La base de datos que constituye el catálogo se reemplaza por una sencilla tabla de objetos. El método `buscaVehiculos` realiza la búsqueda de uno o de varios vehículos en función de su precio gracias a un simple bucle.

```

1 import java.util.*;
2 public interface Catalogo {
3
4     List<String> buscaVehiculos(int precioMin, int precioMax);
5 }
6
7 import java.util.*;
8 public class ComponenteCatalogo implements Catalogo {
9     protected Object[] descripcionVehiculo =
10     {
11         "Berlina 5 puertas", 6000, "Compacto 3 puertas", 4000,
12         "Espace 5 puertas", 8000, "Break 5 puertas", 7000,
13         "Coupe 2 puertas", 9000, "Utilitario 3 puertas", 5000
14     };
15
16     public List<String> buscaVehiculos(int precioMin, int precioMax) {
17         int indice, tamaño;
18         List<String> resultado = new ArrayList<String>();
19         tamaño = descripcionVehiculo.length / 2;
20         for (indice = 0; indice < tamaño; indice++) {
21             int precio = (Integer)descripcionVehiculo[2 * indice + 1];
22             if ((precio >= precioMin) && (precio <= precioMax))
23                 resultado.add((String)descripcionVehiculo[2 * indice]);
24         }
25         return resultado;
26     }
27 }

```

Continuamos con el componente de gestión de documentos constituido por la interfaz `GestionDocumento` y la clase `ComponenteGestionDocumento`. Este componente constituye la simulación de una base de documentos.

```

1 public interface GestionDocumento {
2     String documento(int indice);
3 }
4
5 public class ComponenteGestionDocumento implements GestionDocumento {
6     public String documento(int indice) {
7         return "Documento numero " + indice;
8     }
9 }

```

La interfaz de la fachada llamada `WebServiceAuto` define la firma de dos métodos destinados a los clientes del servicio web.

```

1 import java.util.List;
2 public interface WebServiceAuto {
3     String documento(int indice);
4     List<String> buscaVehiculos(int precioMedio, int desviacionMax);
5 }

```

La clase `WebServiceAutoImpl` implementa ambos métodos. Destacamos el cálculo del precio mínimo y máximo para poder invocar al método `buscaVehiculos` de la clase `ComponenteCatalogo`.

```

1 import java.util.List;
2 public class WebServiceAutoImpl implements WebServiceAuto {
3
4     protected Catalogo catalogo = new ComponenteCatalogo();
5     protected GestionDocumento gestionDocumento = new ComponenteGestionDocumento();
6
7     public String documento(int indice) {
8         return gestionDocumento.documento(indice);
9     }
10    public List<String> buscaVehiculos(int precioMedio, int desviacionMax) {
11        return catalogo.buscaVehiculos(precioMedio - desviacionMax, precioMedio + desviacionMax);
12    }
13 }

```

Por último, un cliente del servicio web puede escribirse en Java como sigue:

```

1 import java.util.*;
2 public class UsuarioWebService {
3
4     public static void main(String[] args) {
5
6         WebServiceAuto webServiceAuto = new WebServiceAutoImpl();

```

```

7      System.out.println(webServiceAuto.documento(0));
8      System.out.println(webServiceAuto.documento(1));
9      List<String> resultados = webServiceAuto.buscaVehiculos(6000, 1000);
10     if (resultados.size() > 0) {
11         System.out.println(
12             "Vehículo(s) cuyo precio esta comprendido "+ "entre 5000 y 7000");
13         for (String resultado: resultados)
14             System.out.println("    " + resultado);
15     }
16 }
17 }

```

Este cliente muestra dos documentos así como los vehículos cuyo precio está comprendido entre 5.000 y 7.000. El resultado de la ejecución de este programa principal es el siguiente:

```

Documento número 0
Documento número 1
Vehículo(s) cuyo precio está comprendido entre 5000 y 7000
Berlina 5 puertas
Break 5 puertas
Utilitario 3 puertas

```

1.4. Decorator

1.4.1. Descripción

El objetivo del patrón **Decorator** es agregar dinámicamente funcionalidades suplementarias a un objeto. Esta agregación de funcionalidades no modifica la interfaz del objeto y es transparente de cara a los clientes.

El patrón **Decorator** constituye una alternativa respecto a la creación de una subclase para enriquecer el objeto.

1.5. Ejemplo

El sistema de venta de vehículos dispone de una clase **VistaCatálogo** que muestra, bajo el formato de un catálogo electrónico, los vehículos disponibles en una página web.

Queremos, a continuación, visualizar datos suplementarios para los vehículos “de alta gama”, a saber la información técnica ligada al modelo. Para agregar esta funcionalidad, podemos crear una subclase de visualización específica para los vehículos "de alta gama". Ahora, queremos mostrar el logotipo de la marca en los vehículos “de gamas media y alta”. Conviene crear una nueva subclase para estos vehículos, súperclase de la clase de vehículos “de alta gama”, lo cual se vuelve rápidamente complejo. Es fácil darse cuenta de que la herencia no está adaptada a lo que se demanda.

El patrón **Decorator** proporciona otro enfoque que consiste en agregar un nuevo objeto llamado decorador que se sustituye por el objeto inicial y que lo referencia. Este decorador posee la misma interfaz, lo cual vuelve a la sustitución transparente de cara a los clientes. En nuestro caso, el método **visualiza** lo intercepta el decorador que solicita al objeto inicial su visualización y a continuación la enriquece con información complementaria.

El patrón **Decorator** proporciona otro enfoque que consiste en agregar un nuevo objeto llamado decorador que se sustituye por el objeto inicial y que lo referencia. Este decorador posee la misma interfaz, lo cual vuelve a la sustitución transparente de cara a los clientes. En nuestro caso, el método **visualiza** lo intercepta el decorador que solicita al objeto inicial su visualización y a continuación la enriquece con información complementaria.

La Figura 7 ilustra el uso del patrón **Decorator** para enriquecer la visualización de vehículos. La interfaz **ComponenteGráficoVehículo** constituye la interfaz común a la clase **VistaVehículo**, que queremos enriquecer, y a la clase abstracta **Decorator**, interfaz constituida únicamente por el método **visualiza**.

La clase **Decorator** posee una referencia hacia un componente gráfico. Esta referencia la utiliza el método **visualiza** que delega la visualización en este componente.

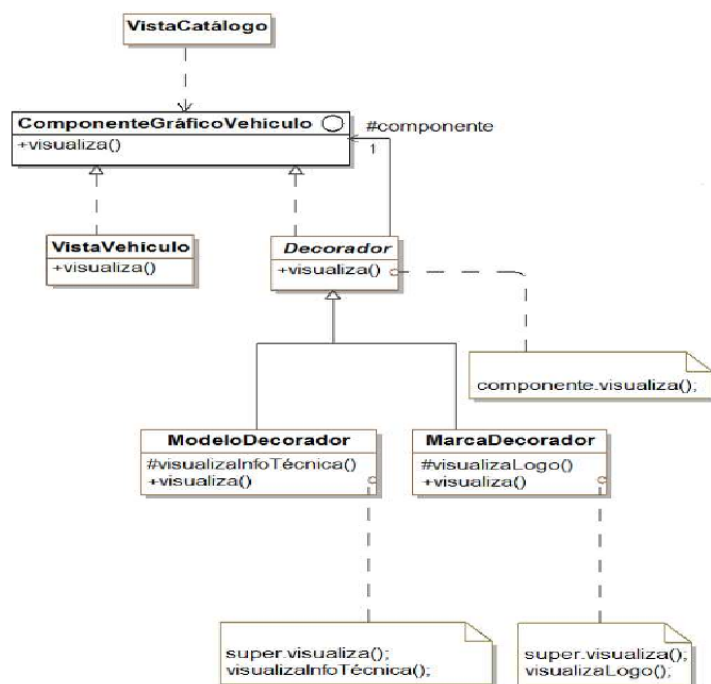


Figura 7: El patrón Decorator para la visualización de vehículos en un catálogo electrónico

Existen dos clases concretas de decorador, subclases de **Decorator**. Su método **visualiza** empieza llamando al método **visualiza** de **Decorator** y a continuación muestra los datos complementarios tales como la información técnica del vehículo o el logotipo de la marca. Los decoradores son componentes puesto que pueden transformar el componente en un nuevo decorador, lo cual da lugar a una cadena de decoradores. Esta posibilidad de encadenar componentes en la que es posible agregar o eliminar dinámicamente un decorador proporciona una gran flexibilidad.

1.5.1. Estructura

La Figura 8 detalla la estructura genérica del patrón.

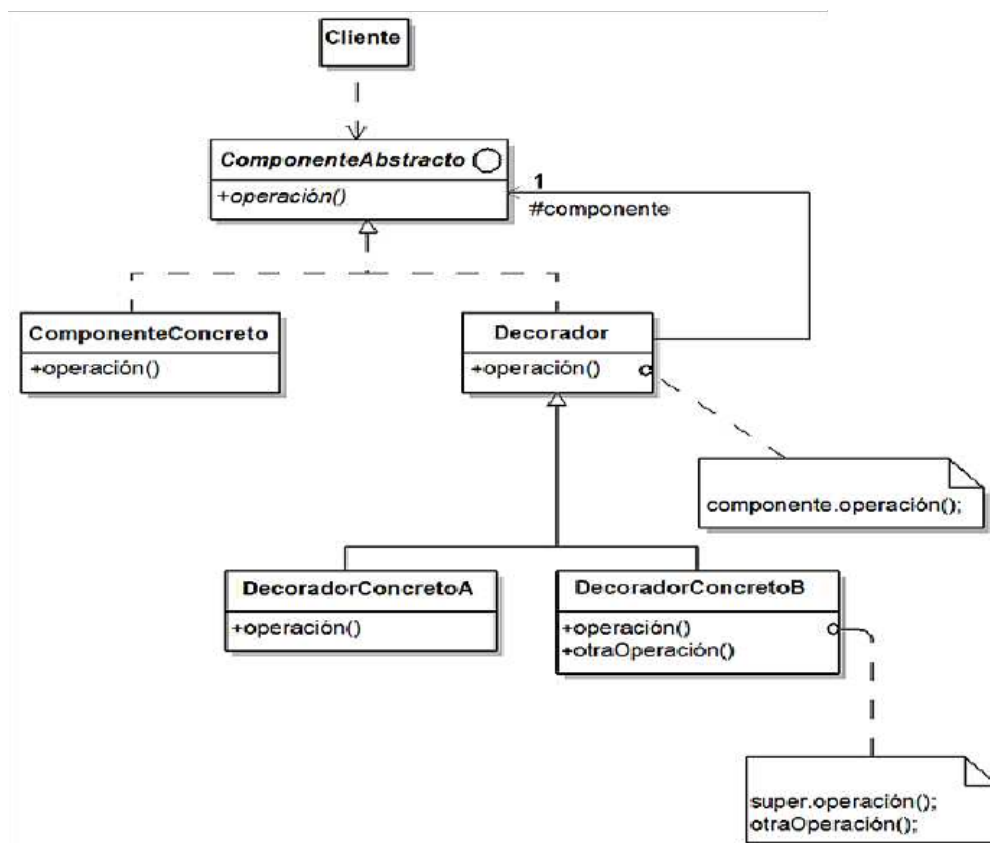


Figura 8: Estructura del patrón Decorator

1.5.2. Participantes

Los participantes del patrón son los siguientes:

- **ComponenteAbstracto** (**ComponenteGráficoVehículo**) es la interfaz común al componente y a los decoradores.
- **ComponenteConcreto** (**VistaVehículo**) es el objeto inicial al que se deben agregar las nuevas funcionalidades.
- **Decorador** es una clase abstracta que guarda una referencia hacia el componente.
- **DecoradorConcretoA** y **DecoradorConcretoB** (**ModeloDecorador** y **MarcaDecorador**) son subclasses concretas de **Decorador** que tienen como objetivo implementar las funcionalidades agregadas al componente.

1.5.3. Colaboradores

El decorador se sustituye por el componente. Cuando recibe un mensaje destinado a este último, lo redirige al componente realizando operaciones previas o posteriores a esta redirección.

1.5.4. Dominios de Aplicación

El patrón Decorator puede utilizarse en los siguientes dominios:

- Un sistema agrega dinámicamente funcionalidades a un objeto, sin modificar su interfaz, es decir sin que los clientes de este objeto tengan que verse modificados.
- Un sistema gestiona funcionalidades que pueden eliminarse dinámicamente.
- El uso de la herencia para extender los objetos no es práctico, lo cual puede ocurrir cuando su jerarquía ya es de por sí compleja.

1.5.5. Ejemplo de Java

Se presenta a continuación el código fuente en Java del ejemplo, comenzando por la interfaz `ComponenteGraficoVehiculo`.

```
1 public interface ComponenteGraficoVehiculo {  
2     void visualiza();  
3 }
```

La clase `VistaVehiculo` implementa el método `visualiza` de la interfaz `ComponenteGraficoVehiculo`.

```
1 public class VistaVehiculo implements ComponenteGraficoVehiculo {  
2     public void visualiza() {  
3         System.out.println("Visualizacion del vehiculo");  
4     }  
5 }
```

La clase `Decorador` implementa a su vez el método `visualiza` delegando la llamada. Tiene un atributo que contiene una referencia hacia un componente. Este último se pasa como parámetro al constructor de `Decorador`.

```
1 public abstract class Decorador implements ComponenteGraficoVehiculo {  
2     protected ComponenteGraficoVehiculo componente;  
3  
4     public Decorador(ComponenteGraficoVehiculo componente) {  
5         this.componente = componente;  
6     }  
7  
8     public void visualiza() {  
9         componente.visualiza();  
10    }  
11 }
```

El método `visualiza` del decorador concreto `ModeloDecorador` llama a la visualización del componente (mediante el método `visualiza` de `Decorador`) y a continuación muestra la información técnica del modelo.

```
1 public class ModeloDecorador extends Decorador {  
2  
3     public ModeloDecorador(ComponenteGraficoVehiculo componente) {  
4         super(componente);  
5     }  
6  
7     protected void visualizaInformacionTecnica() {  
8         System.out.println("Informacion tecnica del modelo");  
9     }  
10  
11     public void visualiza() {  
12         super.visualiza();  
13         this.visualizaInformacionTecnica();  
14     }  
15 }  
16 }
```

El método `visualiza` del decorador concreto `MarcaDecorador` llama a la visualización del componente y a continuación muestra el logotipo de la marca.

```
1 public class MarcaDecorador extends Decorador  
2 {  
3     public MarcaDecorador(ComponenteGraficoVehiculo componente) {  
4         super(componente);  
5     }  
6  
7     protected void visualizaLogo() {  
8         System.out.println("Logotipo de la marca");  
9     }  
10  
11     public void visualiza() {  
12         super.visualiza();  
13         this.visualizaLogo();  
14     }  
15 }  
16 }
```

Por último, la clase `VistaCatalogo` es el programa principal. Este programa crea un vehículo, un decorador de modelo que toma la vista como componente, y un decorador de marca que toma el decorador de modelo como componente. A continuación, este programa solicita la visualización al decorador de marca.

```
1
2 public class VistaCatalogo
3 {
4     public static void main(String[] args)
5     {
6         VistaVehiculo vistaVehiculo = new VistaVehiculo();
7         ModeloDecorador modeloDecorador = new
8         ModeloDecorador(vistaVehiculo);
9         MarcaDecorador marcaDecorador = new
10        MarcaDecorador(modeloDecorador);
11        marcaDecorador.visualiza();
12    }
13 }
```

El resultado es el siguiente:

Visualización del vehículo
Información técnica del modelo
Logotipo de la marca